

PROJECT REPORT

Implement Client Script and UI Policy for Incident
ServiceNow Incident triage | Prepared 30 September 2026

Project status: design and sample artifacts prepared; target-instance implementation and UAT evidence pending.

High Impact Incident Form

Conceptual illustration - not a screenshot from a live ServiceNow instance

INCIDENT | HIGH IMPACT TRIAGE

Impact	1 - High
Urgency	1 - High [read only]
Assignment group	Required
Assigned to	Required
State	Change from the record form

If group or assignee is missing at High Impact, save is blocked with a field message.

Conceptual Incident form - not a live ServiceNow screenshot.

Team: to be completed | Guide: supplied Incident Client Script & UI Policy reference

1. INTRODUCTION

1.1 Project Overview

This project proposes client-side controls for the ServiceNow Incident form so agents can handle high-impact records consistently. The design combines one UI Policy with three Client Scripts and retains the standard Incident table and existing platform security.

1.2 Purpose

The purpose is to guide high-impact triage, reduce avoidable incomplete submissions, and make State changes on the Incident form rather than through direct list editing. Client-side controls improve the form experience; they do not replace access controls or server-side enforcement.

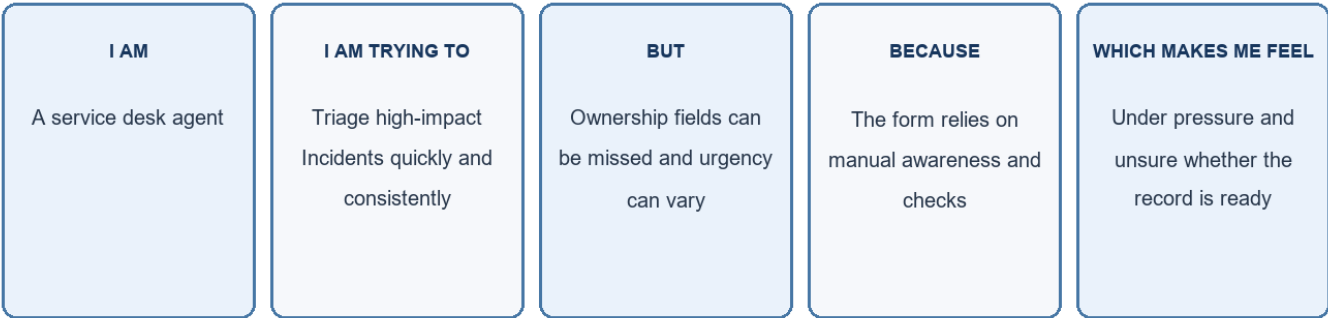
2. IDEATION PHASE

2.1 Problem Statement

Service desk agents need to triage high-impact Incidents quickly, but manual entry can leave urgency or ownership incomplete and a list update can bypass form guidance. The result may be unclear routing, repeated follow-up, and inconsistent record quality. This statement is a working hypothesis to validate with users.

High Impact Incident Problem Statement

Service desk workflow



Problem statement canvas.

2.2 Empathy Map Canvas

The agent persona is under time pressure and values clear required fields, ownership, and feedback. This empathy map is a project hypothesis, not interview findings.

Empathy Map for a Service Desk Agent

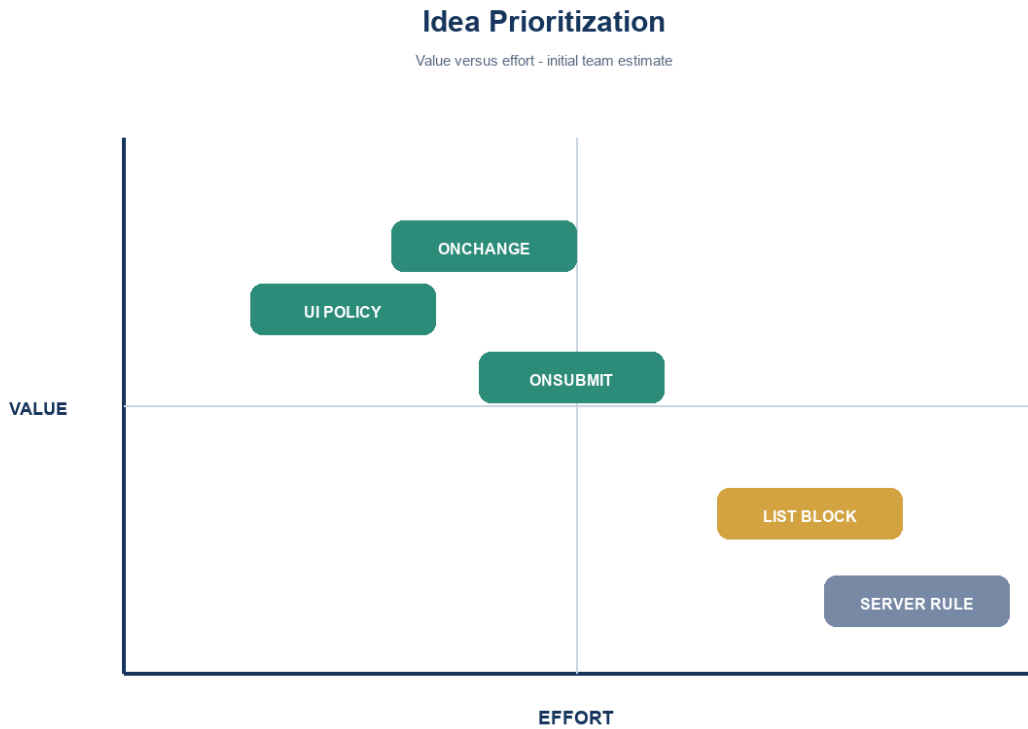
Working hypothesis - validate with real users



Service desk agent empathy map - validate with real users.

2.3 Brainstorming

Ideas considered include a UI Policy for mandatory routing, an Impact onChange script to set Urgency, onSubmit validation with field-level errors, and an onCellEdit guard for list State changes. Server-side validation is a separate future requirement if policy must hold across imports, integrations, or other entry paths.



Initial value-effort ranking; estimates are not measured results.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The journey covers intake, opening the record, assessing impact, routing and saving, and working/closing. The main opportunity is to guide routing at the point of entry.

Incident Triage Customer Journey

Service desk agent - from intake to follow-up

	Receive	Open record	Assess impact	Route and save	Work and close
GOAL	Understand the issue	Capture key details	Set correct severity	Assign clear ownership	Resolve and report
TOUCHPOINT	Email or call	Incident form	Impact and Urgency	Group, assignee, save	Incident record and list
PAIN POINT	Incomplete context	Required fields unclear	Manual urgency entry	Missing owner blocks work	List edit lacks context
OPPORTUNITY	Capture structured facts	Guide the form	Set urgency on change	Require routing and explain errors	Use form for State updates

Journey map hypothesis for review with service desk users.

3.2 Solution Requirement

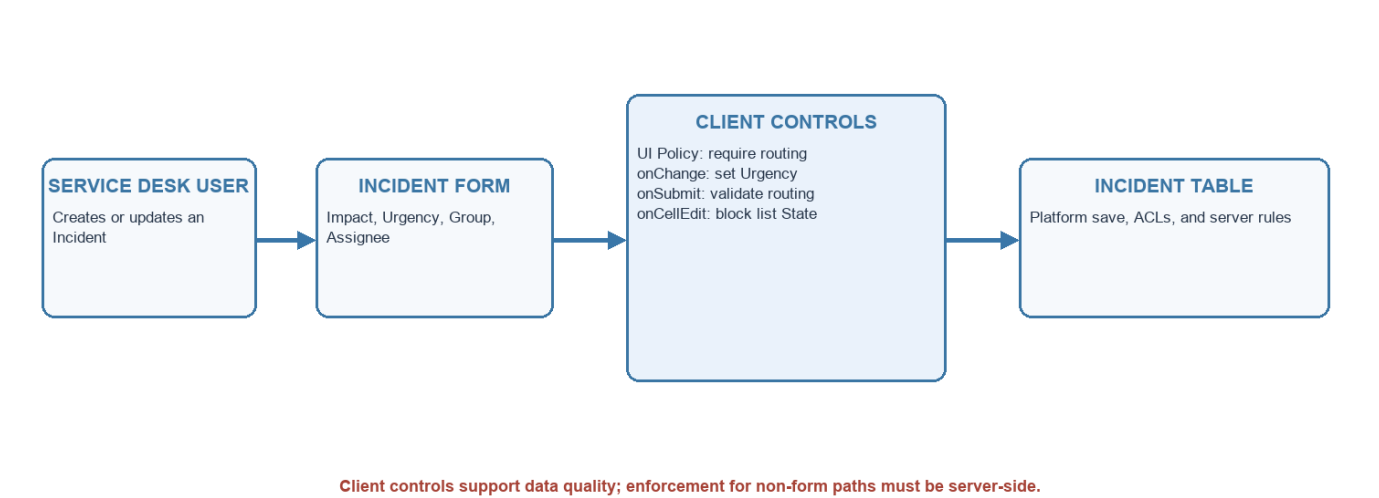
ID	Requirement
FR-1	When Impact=1 (High), require Assignment group and Assigned to; make Urgency read-only.
FR-2	When Impact changes to 1, set Urgency to 1 (High) and inform the user.
FR-3	Before save, block a high-impact Incident missing either routing field and show a field-level message.
FR-4	When Impact is no longer 1, reverse mandatory/read-only policy settings without clearing values.
FR-5	Decline Incident list edits to State and direct users to update it from the form.
NFR-1	Treat client-side behavior as usability guidance; use server-side controls for enforceable policy.

Assumption: standard choice values are Impact=1 for High and Urgency=1 for High. Confirm them in the target instance before activation.

3.3 Data Flow Diagram

Incident High Impact Data Flow

Form inputs move through client controls to the Incident table



Client controls operate in the form; non-form enforcement must be server-side.

3.4 Technology Stack

ServiceNow platform, Incident table, standard desktop form and list, one Incident UI Policy, and three client-side scripts. No external service, dataset, analytics platform, GenAI, or ML model is part of the proposed scope.

4. PROJECT DESIGN

4.1 Problem-Solution Fit

The proposed fit is that visible required fields, automatic urgency selection, save-time validation, and form-based State updates address likely triage pain points. Stakeholder review and UAT are needed to confirm the hypothesis.

Problem Solution Fit

Incident triage - hypotheses for validation

CUSTOMER Service desk agent under time pressure High impact incidents lack consistent routing	CONSTRAINT Manual checks can be missed Existing policies may conflict	CURRENT ALTERNATIVES Checklist, training, manager follow-up Manual data correction after submission
TRIGGERS Impact is set to High User attempts save or list State edit	SOLUTION Require group and assignee; set Urgency High Show clear field errors; guide State update	CHANNELS ServiceNow desktop Incident form Incident list for visibility
BEFORE Unclear ownership and repeated follow-up Variable urgency entry	AFTER More complete triage data Less avoidable correction	SUCCESS SIGNALS Required fields complete on UAT cases Users understand the error and next action

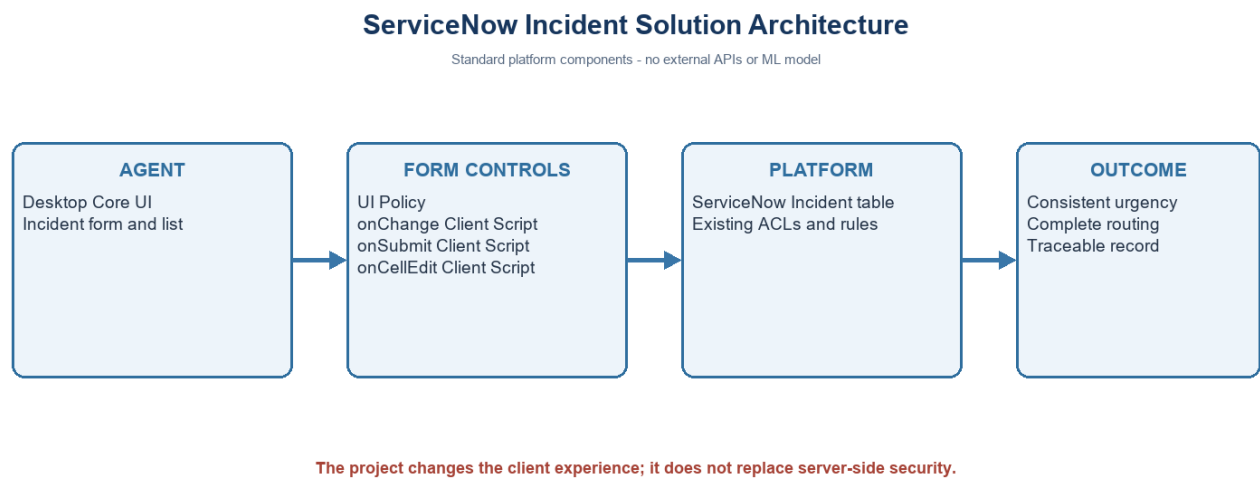
Problem-solution fit canvas.

4.2 Proposed Solution

UI Policy: table Incident; active; on load; condition Impact is 1; Reverse if false enabled. Actions: Assignment group mandatory, Assigned to mandatory, Urgency read-only. Visibility unchanged.

Client Scripts: onChange on Impact sets Urgency=1 when Impact changes to 1; onSubmit checks both routing fields and returns false when either is missing; onCellEdit calls callback(false) for Incident list State edits and allows other table edits.

4.3 Solution Architecture

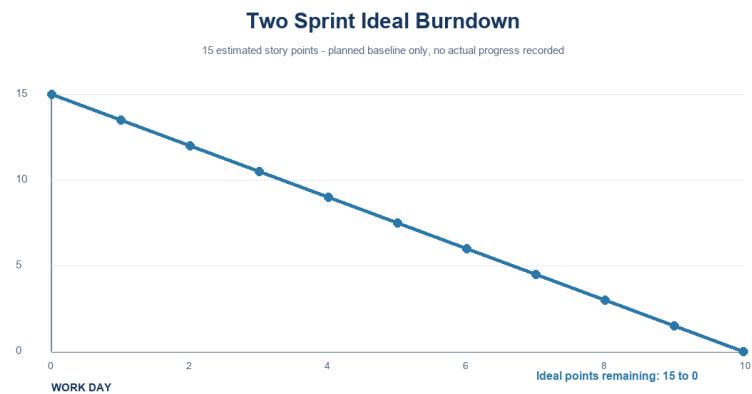


Standard ServiceNow form-to-platform flow.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The planning template contains a two-sprint, ten-working-day estimate totaling 15 story points. This is a planning baseline, not completed work or observed team velocity. Sequence: confirm fields/policies; configure and peer-review; run UAT in a non-production instance; remediate; obtain sign-off.



Illustrative planned burndown only; no actual progress data is represented.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The design makes no external API calls and does not include a model. Functional and performance measurements have not been run. Eight checks are planned: six functional cases for policy behavior, urgency, routing validation, reversal, list State edits, and form State updates; plus two platform response checks.

UAT Status

Planned cases only - not executed in an instance

8 TEST CASES

High Impact policy | onChange urgency | onSubmit routing | reverse condition | list edit | form update | non-high-impact

UAT status: 8 planned, 8 not run, 0 pass, 0 fail.

7. RESULTS

7.1 Output Screenshots

The form image is a conceptual wireframe. It is not evidence from a live instance. No deployment, observed behavior, test result, or user approval is claimed in this report.

8. ADVANTAGES & DISADVANTAGES

Advantages: immediate guidance; predictable high-impact urgency; explicit ownership requirements; clear save-time feedback; direct list State edits declined.

Limitations: client scripts can be bypassed by imports, APIs, integrations, or alternate forms; actual field names and choice values may differ; existing policies may conflict; users may need training; UI-only list blocking does not provide server security.

9. CONCLUSION

The project defines a coherent Incident form experience for high-impact triage and provides configuration notes and script examples. Instance configuration, conflict review, non-production execution, and UAT sign-off remain outstanding.

10. FUTURE SCOPE

If the business rule must apply beyond the UI, assess a Data Policy, Business Rule, or other server-side validation with the platform owner. Add instance-specific field/value checks, accessibility review, user feedback, and operational monitoring.

11. APPENDIX

Source Code: the three JavaScript examples and UI Policy note are included in the Development Phase ZIP. Dataset Link: not applicable; this workflow uses Incident records and no dataset. GitHub & Project Demo Link: to be supplied by the project team if created.

Implementation guide supplied by the project team: Implement Client Script & UI Policy (Incident).docx.

ServiceNow references

Client Scripts: <https://www.servicenow.com/docs/r/api-reference/scripts/client-scripts.html>

Useful client-side scripts: <https://www.servicenow.com/docs/r/api-reference/scripts/useful-client-side-scripts.html>

UI Policy fields: <https://www.servicenow.com/docs/r/application-development/form-builder-glide-family-release/ui-policy-fields.html>